

Applied Artificial Intelligence and Retrieval-Augmented Generation

A Technical Overview of Modern AI Systems

Abstract

Modern artificial intelligence systems combine machine learning, neural networks, natural language processing, information retrieval, vector databases, and generative models. This document presents a structured overview of these components and explains how they interact inside a Retrieval-Augmented Generation (RAG) pipeline. The material is intentionally multi-topic and detailed so that it can be used as a realistic knowledge source for document ingestion, chunking, semantic retrieval, and question answering.

1. Artificial Intelligence and Machine Learning

Artificial intelligence (AI) is a broad field concerned with building systems capable of performing tasks that normally require aspects of human intelligence, such as perception, reasoning, prediction, planning, language understanding, and decision making. Machine learning (ML) is a major approach within AI in which a model learns patterns from data rather than relying entirely on manually written rules.

Supervised learning uses labeled examples to learn a mapping between inputs and target outputs. Classification predicts discrete categories, while regression predicts continuous values. Unsupervised learning works with data where target labels are unavailable and can be used for clustering, dimensionality reduction, and representation learning. Reinforcement learning instead learns through interaction with an environment using rewards or penalties.

2. Deep Learning and Neural Networks

Deep learning uses multilayer neural networks to learn hierarchical representations. A typical neural network contains input units, hidden layers, activation functions, and output units. During training, the model computes a prediction, measures the difference between prediction and target using a loss function, and updates parameters through backpropagation and an optimization algorithm such as stochastic gradient descent or Adam.

Convolutional neural networks are commonly associated with image and spatial data. Recurrent architectures were historically important for sequential data, while Transformer architectures became highly influential for language and multimodal workloads because self-attention can model relationships between tokens over long contexts.

3. Natural Language Processing

Natural language processing (NLP) focuses on computational processing of human language. Common tasks include text classification, named-entity recognition, sentiment analysis, summarization, translation, information extraction, question answering, and semantic search. Modern NLP systems often transform text into numerical representations called embeddings, which allow semantically related text to occupy nearby regions in a vector space.

4. Embeddings and Semantic Similarity

An embedding model converts text into a fixed-length numerical vector. Unlike keyword matching, semantic retrieval can identify related concepts even when the exact words differ. For example, a query about fraudulent transactions may retrieve text discussing financial fraud detection because the meanings are related. Cosine similarity is frequently used to compare normalized embedding vectors.

5. Retrieval-Augmented Generation

Retrieval-Augmented Generation combines information retrieval with text generation. Instead of asking a language model to answer entirely from its learned parameters, the system first retrieves relevant passages from an external knowledge base. Those passages are inserted into a prompt and supplied to the language model as context. This architecture is useful when answers should be grounded in private, domain-specific, or frequently changing documents.

6. Document Ingestion and Chunking

A practical RAG system begins by ingesting documents. PDF files can be parsed page by page, after which the extracted text is divided into smaller chunks. Chunk size is an important design parameter: very small chunks may lose context, while very large chunks can reduce retrieval precision and consume more model context.

Overlapping chunks preserve information near boundaries. A small overlap can help ensure that related ideas spanning a boundary remain retrievable together. More advanced systems may split by headings, paragraphs, sentences, or token counts instead of using only character boundaries.

7. Vector Databases

A vector database stores embeddings together with the original text and metadata. Metadata may include document name, page number, section title, timestamp, or document type. During retrieval, the query is embedded using the same embedding model and compared against stored vectors. The highest-scoring records are returned as candidate context.

ChromaDB is one example of a vector database that can be used for local development. Persistent storage is useful because embeddings do not need to be regenerated every time the application starts. A production deployment may additionally require access control, monitoring, backup policies, collection management, and strategies for updating or deleting stale documents.

8. Retrieval Quality and Evaluation

Retrieval quality can be evaluated independently from answer generation. Useful measures include recall at k, precision at k, and mean reciprocal rank. In a RAG application, evaluation should also consider whether the retrieved passages actually contain evidence needed to answer the question. A high-quality generator cannot reliably compensate for consistently missing or irrelevant retrieved context.

9. Prompt Construction and Grounded Generation

A grounded prompt should clearly separate retrieved context from the user's question. Instructions can tell the model to answer only from supplied evidence and to explicitly state when the available documents do not contain enough information. This reduces unsupported answers, although prompt instructions alone do not guarantee factual correctness.

10. Security and Reliability Considerations

RAG systems can be affected by malicious or misleading documents, prompt injection, sensitive data leakage, incorrect permissions, and stale information. A robust system should validate sources, apply appropriate access controls, log retrieval decisions, and avoid exposing documents that a user is not authorized to read. For high-impact applications, human review and systematic evaluation may be necessary.

11. Example RAG Workflow

Step 1: Load PDF documents. **Step 2:** Extract text and metadata. **Step 3:** Split text into overlapping chunks. **Step 4:** Generate embeddings. **Step 5:** Store chunks and vectors in a vector database. **Step 6:** Convert a user question into

an embedding. **Step 7:** Retrieve the top-k relevant chunks. **Step 8:** Build a grounded prompt. **Step 9:** Generate the final response. **Step 10:** Return the answer together with useful source metadata.

Component	Primary role	Typical output
Document loader	Reads source files	Page-level text
Chunker	Splits long text	Manageable text chunks
Embedding model	Creates semantic vectors	Dense numerical vectors
Vector database	Stores and searches vectors	Relevant chunks
Retriever	Selects context for a query	Top-k passages
Generator / LLM	Produces natural-language response	Final answer

12. Practical Design Checklist

A maintainable RAG project should separate configuration, ingestion, chunking, embeddings, vector storage, retrieval, generation, and application orchestration. It should also keep secrets out of source control, use deterministic document identifiers where possible, provide reproducible setup instructions, and include basic tests for ingestion and retrieval. As the document collection grows, monitoring retrieval quality and latency becomes increasingly important.

Appendix: Example Questions

1. What is the difference between supervised and unsupervised learning?
2. Why are embeddings useful for semantic search?
3. What problem does chunk overlap solve?
4. How does a vector database participate in a RAG pipeline?
5. Why can poor retrieval reduce the quality of a generated answer?
6. What are some security risks in RAG systems?
7. What metadata can be stored with a retrieved document chunk?